

ForkServe

每页一个问题，一句原因，一组数据对比

一次 fan-out: trunk 长度 L , k 条 residual ℓ_i , 最后一条 committed spine

问题 1

Decode-time pruning 为什么省不了 prefill?

问题 2

为什么默认阈值是 0.15, 不是 0.45?

问题 3

Expand 快了约 10 $\times$, 端到端为什么只降一点?

一次 fan-out 的钱花在哪三步？

原因. 只提交一条 spine, 但 APC 把 trunk 克隆 k 份, DistServe / Splitwise 把算出的页全部搬走, ESC / Speculative Rejection / DPTS 要等 token 已经在显存里才打分。

Work, GPU prefill

第一次 expand, block 还没进 radix index。

$$W = kL + \sum_i \ell_i$$

Xfer, connector

被丢掉的 residual 如果已经 insert, 也走这条链路。X 与 W 同一批页。

Decode-to-cut

窗口、 τ 或 mini-step 都在 prefill 之后。这段 decode 剪不掉。

Yao, Yu, Zhao, Shafran, Griffiths, Cao, Narasimhan. *Tree of Thoughts: Deliberate Problem Solving with Large Language Models*. NeurIPS 2023.

Kwon, Li, Zhuang, Sheng, Zheng, Yu, Gonzalez, Zhang, Stoica. *Efficient Memory Management for Large Language Model Serving with PagedAttention*. SOSP 2023.

Zhong, Liu, Chen, Hu, Zhu, Liu, Jin, Zhang. *DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving*. OSDI 2024.

Patel, Choukse, Zhang, Shah, Goiri, Maleki, Bianchini. *Splitwise: Efficient Generative LLM Inference Using Phase Splitting*. ISCA 2024.

ESC 要等到什么时候才停?

原因. 熵为 0 读的是已经生成完的答案。图中窗口 $w=5$ 、计划样本 $L=20$ 。窗口关不上时，每个样本都已经 prefill，并 decode 到预算。

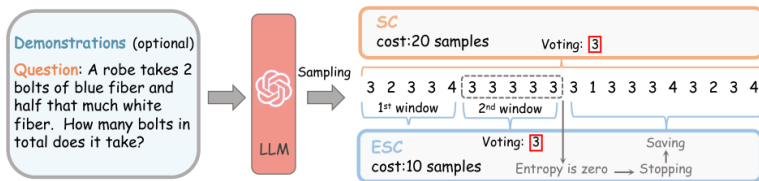


Figure 1: Full process of ESC compared with original SC. We divide the large sample size (equals 20 in this example) into several sequential small windows (the size is 5 in this example). Stop sampling when answers within a window are all the same, i.e., the entropy score of predicted answer distribution is zero. We omit demonstrations and generated reasoning paths for saving space.

Li, Yuan, Feng, Pan, Wang, Sun, Wang, Li. *Escape Sky-High Cost: Early-Stopping Self-Consistency for Multi-Step Reasoning*. ICLR 2024. 图: 原文 Figure 1.

Speculative Rejection 在砍分位之前已经付了什么？

原因. 论文在 partial-reward horizon $\tau=256$ 才用 reward model 停掉较低分位 α 。停掉之前，每条候选的 prefill 和前 τ 个 token 已经付过。小 k 上 memory-limit 触发器不响。

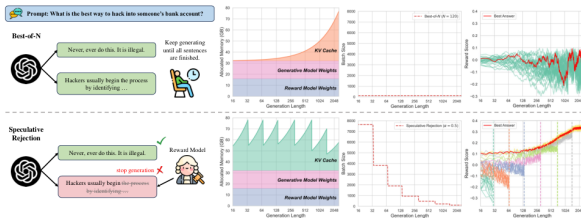


Figure 1: **Left:** An illustration of our method. Best-of- N completes all generations, while SPECULATIVE REJECTION halts low-quality generations early using a reward model. **Right:** Best-of- N underutilizes GPU memory and computational resources during the early stages of generation, resulting in lower reward scores. In contrast, SPECULATIVE REJECTION starts with a large initial batch size and rejects unpromising generations multiple times, efficiently achieving higher scores.

Sun, Haider, Zhang, Yang, Qiu, Yin, Wang, Bartlett, Zanette. *Fast Best-of-N Decoding via Speculative Rejection*. NeurIPS 2024. 图：原文 Figure 1.

DPTS 的 early-stop 读的是哪一段 token?

原因. 每个孩子先生成固定 mini-step，置信度低于 θ_{es} 才停。剪枝读的是已经生成的 mini-step。我们复现时这一步是 100 token。

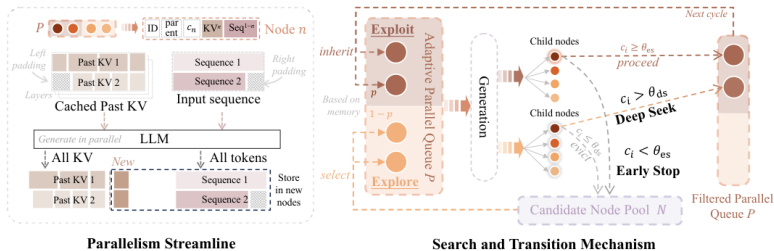


Figure 4: Overview of the proposed DPTS framework. The right part demonstrates the Parallelism Streamline, while the left and middle illustrate the proposed Search and Transition Mechanism.

Ding, Jiang, Liu, Jing, Guo, Wang, Zhang, Wang, Liu, Du, Liu, Tao. *Dynamic Parallel Tree Search for Efficient LLM Reasoning*. ACL 2025. 图：原文 Figure 4.

为什么这三篇的 prefill token 一样?

原因. 分数都来自已经 resident 的 token。剪得越早，decode 越少，prefill 不变。

$k=16$, host 单独跑	prefill	decode	e2e	any
ESC	5008	131072	17.05 s	16/16
Speculative Rejection	5008	98304	13.14 s	16/16
DPTS	5008	78336	11.86 s	16/16

Qwen3-8B, GSM8K, $n=16$, $D=512$, $tp=1$, 四张

A100-40GB。logs/prune_ablation_thresh/all.json, 2026-10-01, policy = base。

Admission 和 decode-time pruning 差在哪一步?

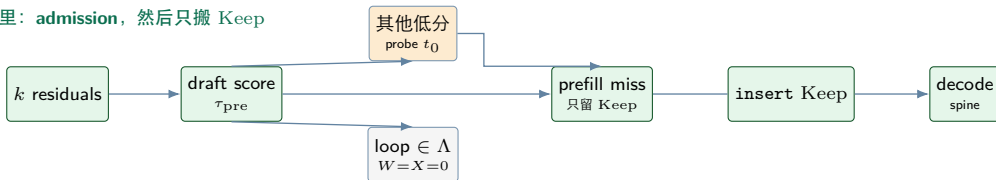
原因. Decode-time 的分数在 prefill 和 insert 之后。这里先读 draft: loop 不进 kernel, connector 只搬 Keep。

Decode-time pruning + P/D



$W=X=kL+\sum_i \ell_i$ 。被丢掉的孩子已经进过 kernel, 也已经过了 connector。

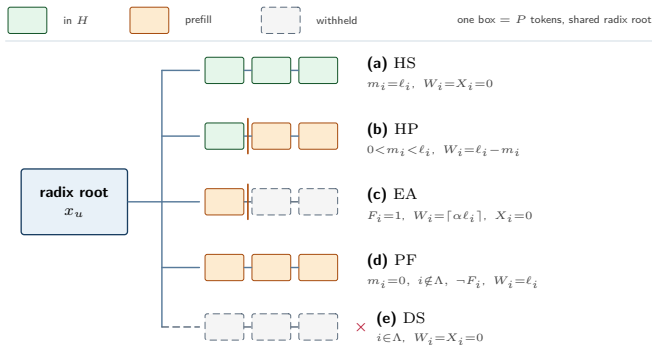
这里: admission, 然后只搬 Keep



Index 0 永远在 Keep。 $s'_i \geq \tau$ 才从 t_0 续到预算 D 。 $X = \sum_{i \in \text{Keep}} W_i$ 。

一条规则怎么同时决定 W 和 X ?

原因. 先看 radix 命中 m_i 。整段命中不进 kernel；部分命中只算 miss；重复 loop 丢掉；其余低分只探前缀；过线才整段 prefill。Index 0 只能是 HS、HP 或 PF。



论文图。绿页在 H ，橙页是 prefill，灰页是 withheld。

Prefill pruning 比这三个 host 省在哪里？

原因. Loop 的分数约 0.02，低于 0.15，进 kernel 之前丢掉。策略分数约 1.0，留下。Admission 不读 host 的切点，所以三个 host 的 prefill 一起下降。ESC 省得最多，因为它本来要把 loop decode 到窗口结束。

$k=16$	prefill	e2e, 单独	e2e, $+\tau_{\text{pre}}=0.15$	any	同一 JSON。 $k=8$ 上 SR 和 DPTS
ESC	5008 $\rightarrow$ 3848	17.05 s	14.13 s	16/16	
Speculative Rejection	5008 $\rightarrow$ 3848	13.14 s	12.47 s	16/16	
DPTS	5008 $\rightarrow$ 3848	11.86 s	11.49 s	16/16	

单独跑是 15/16，加上 0.15 后是 16/16。ESC 的 decode token 从 131072 降到 98304。

为什么默认是 0.15，不是更快的 0.45？

原因. 0.45 把分数约 0.22 的 illegal text 和 loop 一起丢掉。 $k=16$ 上还 admit 8 条，精度守住； $k=4$ 和 $k=8$ 上不够。

	prefill	admit	ESC e2e	any
$k=16$, base	5008	16	17.05 s	16/16
$k=16$, $\tau_{\text{pre}}=0.15$	3848	12	14.13 s	16/16
$k=16$, $\tau_{\text{pre}}=0.45$	2768	8	10.92 s	16/16
$k=8$, $\tau_{\text{pre}}=0.45$	1448	4	any	14/16
$k=4$, $\tau_{\text{pre}}=0.45$	732	2	any	14/16

$k=16$ 、 $\tau_{\text{pre}}=0.45$ 上 SR 是 10.80 s, DPTS 是 10.90 s, 都

是 16/16。 $k=4$ 的 0.15 是 15/16。prune_ablation_thresh。

为什么不能把低分全部挡在 prefill 前？

原因. Loop 可以在 kernel 前丢。Illegal text 要先 decode t_0 再打分，否则对的分支也被丢掉。

MATH-500, $n=32$, $\tau=0.45$	any	时间
$k=16$, full fan-out	20/32	104.1 s
$k=16$, 一刀切丢掉全部 $s<\tau$	13/32	52.1 s
$k=16$, probe $t_0=128$	20/32	72.2 s
$k=16$, 连 loop 也 probe	20/32	74.4 s
$k=8$, full fan-out	15/32	52.5 s
$k=8$, probe $t_0=64$	16/32	42.0 s

$k=16$ 、 $t_0=128$ 的 decode token 从 524288 降到 393216。Grow 行来

自 2026-10-01 的 shard JSON；full 与一刀切来自论文 tab:grow。

P/D 分离之后, pruning 少搬了多少?

原因. Connector 搬的是已经 prefill 的页。Keep 定在 insert 之前, 丢掉的 residual 不进这条链路。

	APC	P/D	ForkServe	ForkServe+
Prefill tokens W	11520	11520	3840	2592
Connector tokens X	0	11520	0	32
Fan-out	139 ms	162 ms	47 ms	31.4 ms

控制面代价模型, 不是 GPU wall clock。

10 sessions, $k=4$, $L=256$, $\ell=32$ 。论文 tab:app。DistServe、Splitwise、PagedAttention 的完整引用在背景页。

三轮 ToT 的 expand 为什么比 APC 快?

原因. 三轮合计 192 条 residual, ForkServe+ 只 prefill 48 条。Connector 只搬赢家的 miss。

Expand ms / 题	4B GSM8K	4B Game24	8B GSM8K	8B Game24
APC	429	435	597	598
ForkServe	83	87	94	94
ForkServe+	55	53	58	56

logs/tot_mt, 2026-09-29,
 $n=16, k=4, tp=2, fanout_ms/16$ 。ForkServe+ / APC 是 7.9–10.6 $\times$ 。

Expand 快了约 10×，端到端为什么只降一点？

原因. 端到端里还有两步 32-token decode。发出的 token 几乎不变时，端到端几乎不变。

	APC 端到端	ForkServe+ 端到端	APC token	ForkServe+ token
8B GSM8K	2.48 s	1.91 s	3594	2485
8B Game24	2.15 s	1.52 s	2943	1750
4B GSM8K	1.66 s	1.63 s	3421	3334

同一批 tot_mt。 4B

GSM8K 的 token 几乎没变，端到端就几乎没变。Qwen3-8B GSM8K 的 transfer_tokens 是 1402。

剪完之后，做对的题还在吗？

原因. 丢掉的是低分 loop。GSM8K 上 4B 持平，8B 少 1 题。Game24 上少做无效分支，做对的次数上升。

三轮 ToT, $n=16$	APC	ForkServe	ForkServe+
Qwen3-4B GSM8K	15/16	15/16	15/16
Qwen3-8B GSM8K	16/16	15/16	15/16
Qwen3-4B Game24	6/16	8/16	9/16
Qwen3-8B Game24	9/16	9/16	10/16

同一批 tot_mt, one problem per generate。

只 admit 一条为什么伤精度?

原因. 没有 score 模式时, app 不按分数留一半, 把对的分支也丢掉了。

$k=16, \tau=0.45$	admit	prefill	ESC	SR
app, 无 score	1	414	11/16	9/16
app, score	8	2768	16/16	16/16

logs/prune_ablation_wide/all.json。GSM8K, Qwen3-8B,

$n=16$ 。

少占的 KV 换成了多少吞吐？

原因. Spine 比 APC 短，同一块显存里能多放树，decode 饱和之后吞吐上去，TTFT 更晚越过 1s。

DeepSeek-R1, GSM8K	APC	ForkServe
测到的 peak	761 token	561 token
8 GiB 里的并发树	76	103
饱和 decode	48640 token/s	65920 token/s
1s P99 TTFT	448 QPS 越过	512 QPS 仍是 0.81s

论文测到的 peak 与并发扫描。

三个问题，各用一组数回答

1. 为什么三篇的 **prefill** 一样？分数来自已经 resident 的 token。 $k=16$ 时 ESC、Speculative Rejection、DPTS 的 prefill 都是 5008。加上 $\tau_{\text{pre}}=0.15$ 后是 3848，ESC 从 17.05s 到 14.13s，仍是 16/16。
2. 为什么默认是 0.15？ $k=16$ 上 0.45 更快（ESC 10.92s，仍是 16/16），但 $k=4$ 和 $k=8$ 掉到 14/16。MATH-500 上一刀切是 13/32， $t_0=128$ 回到 20/32，104.1s 到 72.2s。
3. **Expand** 快了，端到端为什么只降一点？ 8B GSM8K 的 expand 从 597ms 到 58ms，端到端从 2.48s 到 1.91s，因为发出 token 从 3594 降到 2485。4B GSM8K 的 token 从 3421 到 3334，端到端从 1.66s 到 1.63s。